Numpy Practice Exercise #3

Setup

```
B = np.random.randint(-50, 150, size=(7,9))
```

Part 1: Controlled extraction

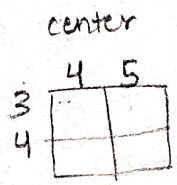
1. Extract a checkerboard-style submatrix:
   - every 2nd row
   - every 2nd column

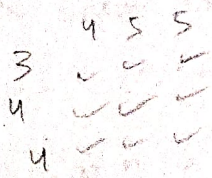
```
B[0::2, 0::2]
```

2. Extract rows where the first element of the row is negative

```
a = B[:,0]
B[a<0]
```

3. From the array, create a 3×3 block from the exact center (think carefully about odd dimensions)






```
a = np.array([4, 5, 5])
b = np.array([3, 4, 4])

B[a, b[:, None]]
```

4. Extract the two rows with the largest row sums

```
a = np.sum(B, axis=1)
b = a[5:]
B[b]
```

Part 2: Logical Filtering

5. Replace all negative number with 0

```
B[B<0]=0
```

6. Extract values that are:

- greater than the column mean of their column

```
a= np.mean(B, axis=0)
B[B>a]
```

7. Count how many values are:

- above the overall mean
- but below the row mean of their row.

```
a=np. mean (B, axis=1)
0= np. mean(B)

B[ (B>a) & (B>0)].shape[0]
```

8. Create a mask that marks the maximum value of each row as True

```
a=np. argsort ( B, axis=1)
b= a[:,-1]
c= B[ np.arange (7), b]

B == c
```

7×9 7×0

Part 3: Sorting with strategy

9. Sort each row in descending order

```
a= np.argsort(B, axis=1)[:,::-1]

B[np.arange (7) [:,None], a]
```

10. Reorder rows based on their minimum value (ascending)

```
a = np.argsort(B, axis=1)[:,0]
b = np.argsort(a)
B[b,:]
```

11. Reorder columns based on their standard deviation

```
a = np.std(B, axis=0)
b = np.argsort(a)
B[:,b]
```

12. without flattening the array:
- get the 5 largest elements in the entire matrix

```
a=np.sort(B, axis=1)
b= np.sort(a, axis=0)
b[-1, -5:-1:-1]
```

Part 4: Shape Thinking

13 For each column:
- subtract the column median from that column

7x9 a,

```
a = np.meadian(B, axis=0)
b= a.reshape(-1)
c = B - b.reshape((1,9))
```

7x9 1x9

14. Normalize each raw so that
- the row sum becomes 1

```
a=np.sum(B, axis=1)[:, None]
b= B / a
```

15.

```
total = B.shape[0]*B.shape[1]
30percent = np.int64( 30/100 * total)
d = np.sort(data.reshape(-1))
be = d[-30percent]  ->  e = np.bincount(d[-30percent:])[0]
B[B<be]=0

row,col = np.nonzero(B==be)

try:
    B[row[e:],col[e:]]=0
except:
```

Numpy practice exercise #3 continued

16. Keep only rows where:
   - the maximum value in the row is greater than the average of all row maximums

```
m= np.max(B, axis=1)
avg= np.mean(m)
B[m>avg]
```

17. Reorder columns so that:
   - columns with more even numbers appear first

```
a= np.sum(B%2==0, axis=0)
b= np.argsort(a)[-1::-1]
B[:,b]
```